

IC TRAINING CENTER VIET NAM
FUNDAMENTAL IC DESIGN AND VERIFICATION COURSE



FINAL PROJECT
TIMER IP DESIGN SPECIFICATION

Student: Hoàng Ngọc Gia Bảo

Class: IC36

Instructor: Phạm Thiều Khang

Contents

LIST OF TABLES	3
LIST OF FIGURES	3
1. Overview	4
1.1. Introduction	4
1.2. Main features	5
1.3. Block diagram	5
1.3.1. APB Slave Interface	6
1.3.2. Register File	6
1.3.3. 64-bit Counter	7
1.3.4. Interrupt Controller	7
1.4. Interface signals	7
2. Register Specification	8
2.1. Register Summary	8
2.2. Timer Control Register (TCR)	8
2.3. Timer Data Register 0 (TDR0)	9
2.4. Timer Data Register 1 (TDR1)	10
2.5. Timer Compare Register 0 (TCMP0)	10
2.6. Timer Compare Register 1 (TCMP1)	11
2.7. Timer Interrupt Enable Register (TIER)	11
2.8. Timer Interrupt Status Register (TISR)	12
2.9. Timer Halt Control Status Register (THCSR)	12
3. Functional Description	13
3.1. APB slave	13
3.1.1. Standard-level behaviour	13
3.1.2. Advanced-level behaviour	14
3.2. Register file	16
3.2.1. Timer Control Register (TCR)	16
3.2.2. Data and Compare Registers (TDR0, TDR1, TCMP0, TCMP1)	17
3.2.3. Timer Interrupt Enable Register (TIER)	22

3.2.4. Timer Interrupt Status Register (TISR).....	22
3.2.5. Timer Halt Control / Status Register (THCSR)	23
3.3. Counter control	23
3.4. 64-bit counter	25
3.5. Interrupt.....	26
3.6. Halt mode (Advanced level)	27
4. History	28

LIST OF TABLES

Table 1: Interface signals of timer IP

Table 2: Register Summary

Table 3: Timer Control Register (TCR)

Table 4: Timer Data Register 0 (TDR0)

Table 5: Timer Data Register 1 (TDR1)

Table 6: Timer Compare Register 0 (TCMP0)

Table 7: Timer Compare Register 1 (TCMP1)

Table 8: Timer Interrupt Enable Register (TIER)

Table 9: Timer Interrupt Status Register (TISR)

Table 10: Timer Halt Control Status Register (THCSR)

Table 11: APB prohibited accesses and error response (Advanced level)

Table 12: Document History Summary

LIST OF FIGURES

Figure 1: Block diagram of Timer IP

Figure 2: APB slave timing - Standard level (zero wait state)

Figure 3: APB slave timing - Advanced level (wait state + error response)

Figure 4: APB slave logic diagram (standard + advanced)

Figure 5: Logic diagram - TCR

Figure 6: Logic diagram TDR0

Figure 7: Logic diagram TDR1

Figure 8: Logic Diagram TCMP0

Figure 9: Logic Diagram TIER

Figure 10: Logic Diagram TISR

Figure 11: Logic Diagram THCSR

Figure 12: Logic diagram - counter control

Figure 13: Logic diagram - 64-bit counter (TDR0/TDR1)

Figure 14: Logic diagram - interrupt

Figure 15: Logic diagram - halt control

1.Overview

1.1. Introduction

A timer is an essential module for every chip. It is used to create exact time delays or to control the timing of different actions in the circuit. This Timer IP is customized from the CLINT (Core Local Interruptor) module of the industrial RISC-V architecture.

Application: Using this timer to make pulses, delays, events, PWM signals, and interrupts based on user settings. The interrupt in this timer is a hardware, maskable, and level interrupt.

Reference: [CLINT chapter of the Chromite M SoC Manual](#)

1.2. Main features

Standard level features:

- A 64-bit count-up counter, presented to software as two 32-bit registers (TDR0 = lower half, TDR1 = upper half).
- Configured through an APB slave interface with a 12-bit address and 32-bit read/write data; the IP is an APB slave.
- APB 32-bit transfers only, with zero wait states and no error handling.
- Active-low asynchronous reset (sys_rst_n).
- Programmable counting speed: the counter can advance at the system-clock rate or be divided by up to 256 using div_en and div_val.
- A maskable, level timer interrupt that can be enabled or disabled, with a pending bit and a write-1-to-clear mechanism.
- On a timer_en High-to-Low transition the counter stops; the testbench (software) re-initialises TDR0/TDR1 before the next run.

Advanced level features (this IP implements all of the advanced-level features in addition to the standard ones):

- One wait state (1 cycle) on APB transfers, signalled by tim_pready.
- Error response (tim_pslverr) for prohibited accesses; when an error occurs, the targeted register is not modified.
- Byte access through tim_pstrb, so individual bytes of a register can be written.
- Hardware clear of the counter: when timer_en goes from High to Low, the counter (TDR0/TDR1) is automatically cleared to its initial value by the IP itself.
- Halt (stop) in debug mode: when dbg_mode is High and halt_req is set, the counter is halted and halt_ack is asserted; counting resumes when the halt request is cleared.

1.3. Block diagram

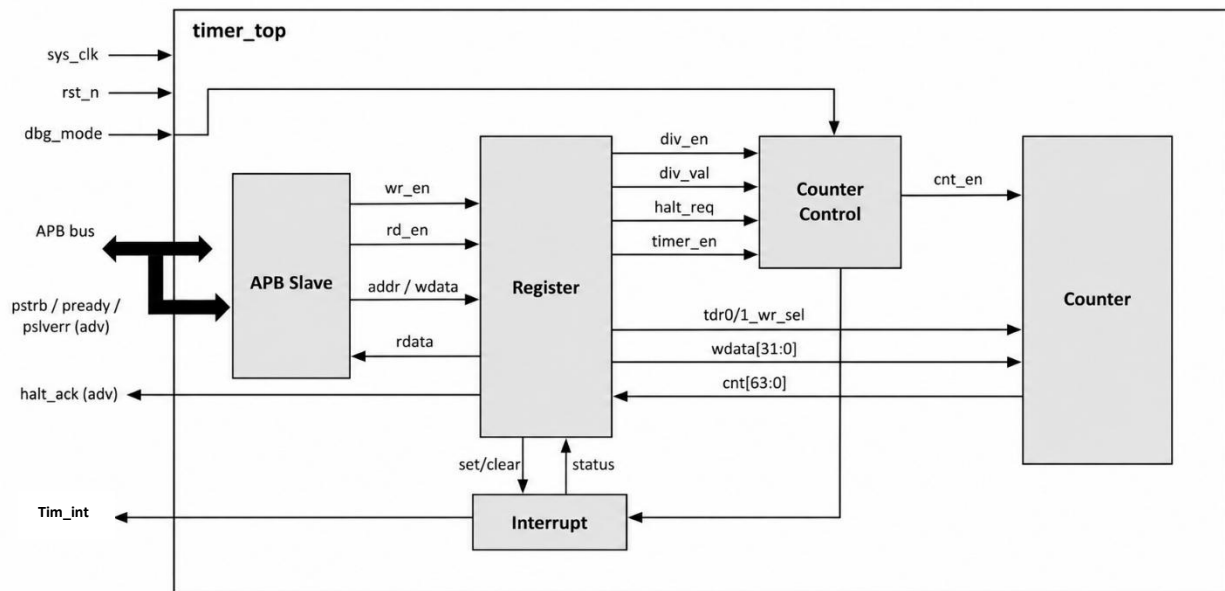


Figure 1: Block diagram of Timer IP

1.3.1. APB Slave Interface

- **Description:** Connects the system APB bus to the inside of the Timer IP. It receives the standard APB signals: address (tim_paddr), control signals (tim_psel, tim_penable, tim_pwrite) and write data (tim_pwdata).
- **Purpose:** It translates APB bus phases into simple internal read/write actions (wr_en, rd_en, reg_addr, reg_wdata) for the register file, and drives reg_rdata back onto tim_prdata. At advanced level it also handles tim_pstrb for byte access, tim_pready for the one-cycle wait state, and tim_pslverr to report a prohibited access.

1.3.2. Register File

- **Description:** the storage of the IP. It holds all user-configurable registers (TCR, TDR0/1, TCMP0/1, TIER, TISR and THCSR).
- **Purpose:** it stores the configuration written by the user and distributes control signals (div_val, div_en, timer_en, int_en, cmp_val, halt_req) to the other blocks. It also captures live hardware status (counter_val, int_st, halt_ack) so the CPU can read it back.

1.3.3. 64-bit Counter

- **Description:** the core of the IP. It takes the system clock and the control signals (timer_en, div_en, div_val) from the register file plus the external dbg_mode signal, and outputs the 64-bit counter_val.
- **Purpose:** it acts as both speed controller and counter. The clock-divide logic uses div_en and div_val to decide exactly when to count up.
- **Hardware clear (Advanced):** when timer_en drops from High to Low, the block resets the counter value to 0.
- **Debug / Halt (Advanced):** it monitors dbg_mode and halt_req; when both are active it safely freezes counting and returns halt_ack.

1.3.4. Interrupt Controller

- **Description:** receives the live counter_val from the counter and the compare value cmp_val from the register file, together with the interrupt enable (int_en) and clear (clr_int) signals.
- **Purpose:** it continuously compares the counter value with the compare value. On an exact match it sets the internal pending flag (int_st). If the interrupt is enabled (int_en = 1) it drives the output tim_int. The flag is reset by the write-1-to-clear command.

1.4. Interface signals

Signal name	Width	Direction	Description
sys_clk	1	Input	System clock.
sys_rst_n	1	Input	Active-low asynchronous reset.
tim_psel	1	Input	APB select signal.
tim_pwrite	1	Input	APB write enable (1 = write, 0 = read).
tim_penable	1	Input	APB enable signal (marks the ACCESS phase).
tim_paddr[11:0]	12	Input	APB address bus.
tim_pwdata[31:0]	32	Input	APB write data bus.
tim_prdata[31:0]	32	Output	APB read data bus.
tim_pstrb[3:0]	4	Input	APB write strobe, used for byte access (Advanced level).

tim_pready	1	Output	APB ready signal, used for the wait state (Advanced level).
tim_pslverr	1	Output	APB slave error, used to report a prohibited access (Advanced level).
tim_int	1	Output	Timer interrupt output signal (level, maskable).
dbg_mode	1	Input	Debug mode. The testbench must not change it while the timer is running (timer_en High).

Table 1: Interface signals of timer IP

2. Register Specification

2.1. Register Summary

Offset	Abbreviation	Register name
0x00	TCR	Timer Control Register
0x04	TDR0	Timer Data Register 0
0x08	TDR1	Timer Data Register 1
0x0C	TCMP0	Timer Compare Register 0
0x10	TCMP1	Timer Compare Register 1
0x14	TIER	Timer Interrupt Enable Register
0x18	TISR	Timer Interrupt Status Register
0x1C	THCSR	Timer Halt Control Register
Others	Reserved	

Table 2: Register Summary

2.2. Timer Control Register (TCR)

Offset address: 0x0

Reset value: 0x0000_0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				DIV_VAL[3:0]				Reserved				DIV_EN		TIM_EN	
				RW								RW		RW	

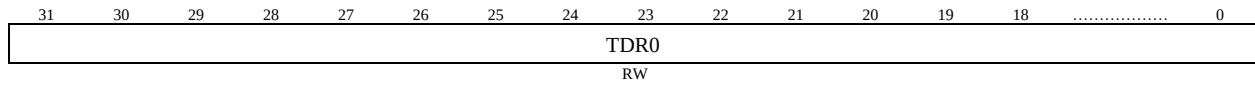
Bit	Name	Type	Default value	Description
31:12	Reserved	RO	20'h0	Reserved.
11:8	div_val[3:0]	RW	4'b0001	Counter control mode setting counting speed: - 4'b0000: Speed is not divided - 4'b0001: Speed is divided by 2 (default) - 4'b0010: Speed is divided by 4 - 4'b0011: Speed is divided by 8 - 4'b0100: Speed is divided by 16 - 4'b0101: Speed is divided by 32 - 4'b0110: Speed is divided by 64 - 4'b0111: Speed is divided by 128 - 4'b1000: Speed is divided by 256 Other: Reserved, div_val not change Writing a prohibited value leaves div_val unchanged; at advanced level the access returns an error response. div_val must not be changed while timer_en is High.
7:2	Reserved	RO	6'b0	Reserved.
1	div_en	RW	1'b0	Counter control-mode enable. 0 = counter counts at the system-clock rate; 1 = counting speed is controlled by div_val. div_en must not be changed while timer_en is High (error response at advanced level).
0	timer_en	RW	1'b0	Timer enable. 0 = counter stopped; 1 = counter counting. On a High-to-Low transition the counter (TDR0/1) is initialised: by software at standard level, by hardware at advanced level.

Table 3. Timer Control Register (TCR)

2.3. Timer Data Register 0 (TDR0)

Offset address: 0x04

Reset value: 0x0000_0000



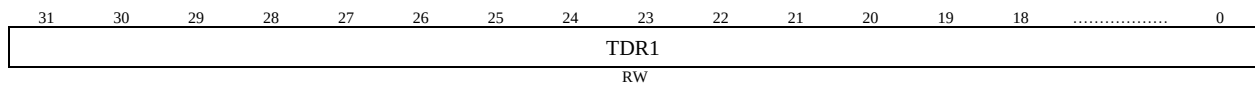
Bit	Name	Type	Default value	Description
31:0	TDR0	RW	32'h0000_0000	Lower 32 bits of the 64-bit counter. A read returns the live counter value. A write sets the initial value. At advanced level this register is cleared to its initial value when timer_en changes from High to Low.

Table 4. Timer Data Register 0 (TDR0)

2.4. Timer Data Register 1 (TDR1)

Offset address: 0x08

Reset value: 0x0000_0000



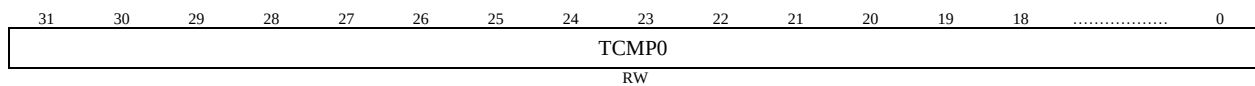
Bit	Name	Type	Default value	Description
31:0	TDR1	RW	32'h0000_0000	Upper 32 bits of the 64-bit counter. A read returns the live counter value. At advanced level this register is cleared to its initial value when timer_en changes from High to Low.

Table 5. Timer Data Register 1 (TDR1)

2.5. Timer Compare Register 0 (TCMP0)

Offset address: 0x0C

Reset value: 0xFFFF_FFFF



Bit	Name	Type	Default value	Description
31:0	TCMP0	RW	32'hFFFF_FFFF	The lower 32 bits of the 64-bit compare value. The interrupt pending bit is set when the counter value equals this compare value.

Table 6. Timer Compare Register 0 (TCMP0)

2.6. Timer Compare Register 1 (TCMP1)

Offset address: 0x10

Reset value: 0xFFFF_FFFF

31	30	29	28	27	26	25	24	23	22	21	20	19	18		0
TCMP1															
RW															

Bit	Name	Type	Default value	Description
31:0	TCMP1	RW	32'hFFFF_FFFF	The upper 32 bits of the 64-bit compare value.

Table 7. Timer Compare Register 1 (TCMP1)

2.7. Timer Interrupt Enable Register (TIER)

Offset address: 0x14

Reset value: 0x0000_0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
RO															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved															Int_en
RO															R/W

Bit	Name	Type	Default value	Description
31:1	Reserved	RO	31'h0	Reserved.
0	int_en	RW	1'b0	Timer interrupt enable. 0 = interrupt disabled (no interrupt is output). 1 = interrupt enabled; tim_int can be driven when the pending bit is set. Writing 0 while an interrupt is asserted

				masks tim_int to 0 but does not affect the pending bit (TISR.int_st).
--	--	--	--	---

Table 8. Timer Interrupt Enable Register (TIER)

2.8. Timer Interrupt Status Register (TISR)

Offset address: 0x18

Reset value: 0x0000_0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
RO															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved															Int_st
RO															RW1C

Bit	Name	Type	Default value	Description
31:1	Reserved	RO	31'h0	Reserved.
0	int_st	RW1C	1'b0	Timer interrupt pending bit. 0 = no interrupt; 1 = the counter value reached the compare value. Write 1 to clear it to 0 (write-1-to-clear); writing 0 has no effect. The clear condition has the highest priority. When the trigger condition occurs the counter keeps counting normally.

Table 9. Timer Interrupt Status Register (TISR)

2.9. Timer Halt Control Status Register (THCSR)

Offset address: 0x1C

Reset value: 0x0000_0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
RO															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved														Halt_ack	Halt_req
RO														RO	RW

Bit	Name	Type	Default value	Description
31:2	Reserved	RO	30'h0	Reserved.
1	halt_ack	RO	1'b0	Timer halt acknowledge (Advanced level). 0 = timer not halted; 1 = timer halted. The timer accepts a halt request only in debug mode (dbg_mode = 1). At standard level this bit is reserved.
0	halt_req	RW	1'b0	Timer halt request (Advanced level). 0 = no request; 1 = request to halt the timer. At standard level this bit is a normal R/W bit with no associated function.

Table 10. Timer Halt Control Status Register (THCSR)

3. Functional Description

3.1. APB slave

The Timer IP has an APB slave that receives the configuration from the user/CPU. It has a 12-bit address bus and a 32-bit read/write data bus.

Every transfer follows the two APB phases: a SETUP phase (tim_psel = 1, tim_penable = 0) and an ACCESS phase (tim_psel = 1, tim_penable = 1).

3.1.1. Standard-level behaviour

At standard level the slave supports only 32-bit transfers with zero wait states and no error response. The internal write strobe wr_en is a single-cycle pulse asserted during the ACCESS phase of a write (tim_psel and tim_penable and tim_pwrite). It tells the register file to store tim_pwdata into the addressed register. Similarly, rd_en is a single-cycle pulse asserted during the ACCESS phase of a read; the read data selected from the register file is driven onto tim_prdata in the same cycle. Reads or writes to a reserved area are RAZ/WI (read as zero, write ignored).

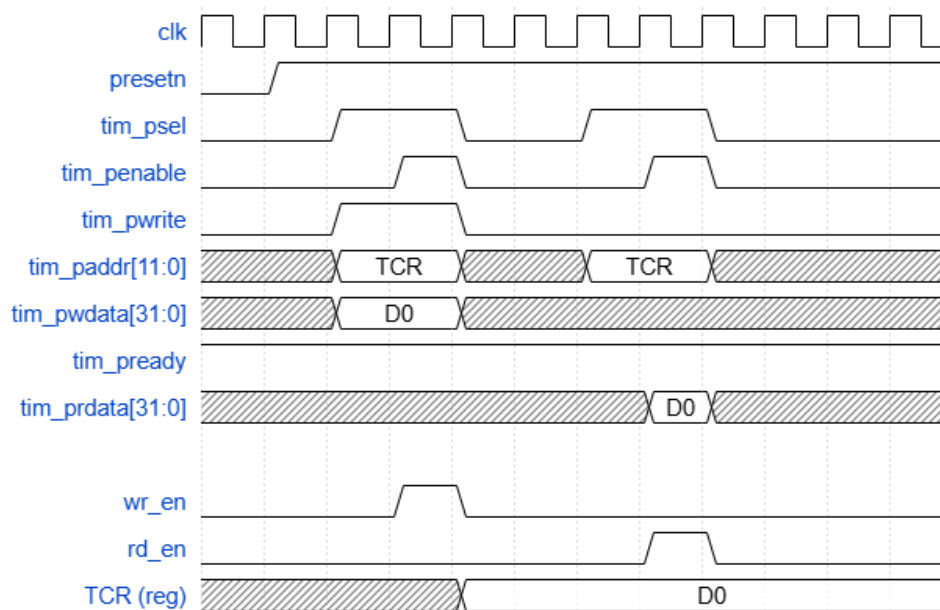


Figure 2: APB slave timing - Standard level (zero wait state)

- `wr_en` is a one-cycle pulse generated in the ACCESS phase of the write transfer (address TCR, data D0); the register updates the following cycle.
- The read transfer asserts `rd_en` and returns D0 on `tim_prdata` in the same ACCESS cycle. Because there is no wait state, `tim_pready` is tied High.

3.1.2. Advanced-level behaviour

At advanced level the slave adds three capabilities: a one-cycle wait state, an error response, and byte access.

- **Wait state:** `tim_pready` is deasserted for one cycle in the ACCESS phase to extend the transfer by one clock; the transfer commits (`wr_en` pulses or read data becomes valid) when `tim_pready` returns High. This relaxes the internal timing.
- **Error response:** `tim_pslverr` is asserted (together with `tim_pready`) for a prohibited access. When an error is reported, `wr_en` is gated off so the target register bit/field is not written.
- **Byte access:** `tim_pstrb[3:0]` qualifies which bytes of `tim_pwdata` are written, so software can update individual bytes of a register.

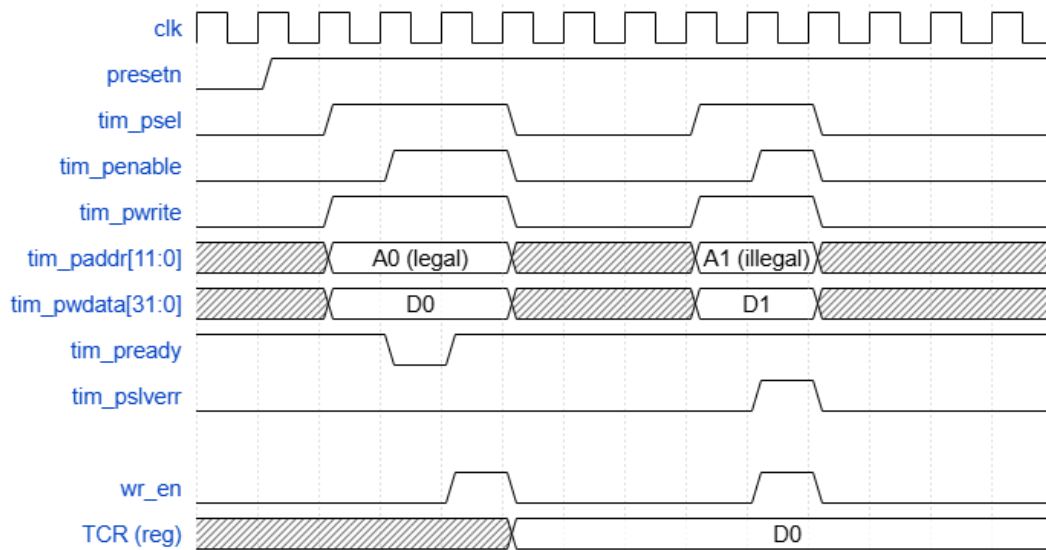


Figure 3: APB slave timing - Advanced level (wait state + error response)

In Figure 3 the first transfer (a legal write to A0) inserts one wait cycle: tim_pready is Low for one cycle, then High, at which point wr_en pulses and the write commits. The second transfer attempts a prohibited write to A1 (e.g. div_val while timer_en is High); the slave asserts tim_pslverr, the write is suppressed, and the register is left unchanged.

The prohibited accesses that produce an error response are summarised in Table 11.

Prohibited access	Result
Write a reserved / prohibited value (0x9 - 0xF) to TCR.div_val	tim_pslverr = 1; div_val unchanged.
Write TCR.div_en while timer_en is High	tim_pslverr = 1; div_en unchanged.
Write TCR.div_val while timer_en is High	tim_pslverr = 1; div_val unchanged.
Access to a reserved address	RAZ/WI; access returns an error response.

Table 11: APB prohibited accesses and error response (Advanced level)

The logic that implements both levels is shown in Figure 4. The standard core (black) generates wr_en and rd_en (combinational decode of psel/penable/pwrite), decodes the address to the per-register write selects, and muxes the addressed register onto tim_prdata. The advanced additions (red) are the wait-state FSM that produces tim_pready, the legality check that produces tim_pslverr and suppresses the write, and the per-byte write-enable path derived from tim_pstrb.

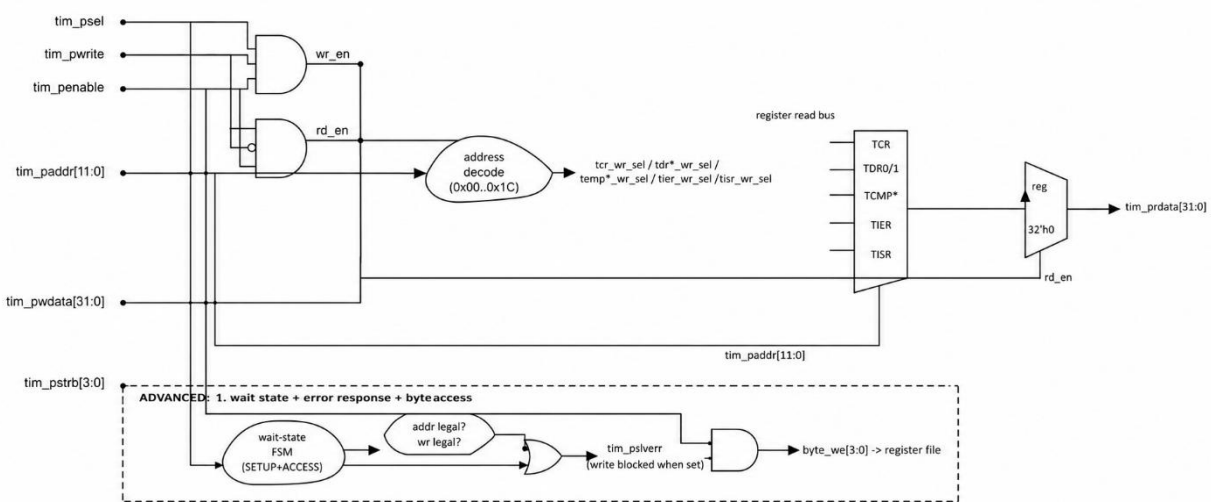


Figure 4: APB slave logic diagram (standard + advanced)

3.2. Register file

The register block stores all configuration and status registers and serves APB read/write accesses. A write uses the decoded address (and, at advanced level, the byte enables from tim_pstrb) to select the target register; a read returns the addressed register through the read multiplexer. The control fields are distributed to the other blocks: timer_en, div_en, div_val and halt_req to the counter side, and int_en, cmp_val (= {TCMP1, TCMP0}) and clr_int to the interrupt controller.

3.2.1. Timer Control Register (TCR)

Holds timer_en, div_en and div_val. Each field is a hold/load flip-flop selected by tcr_wr_sel (= wr_en AND addr = 0x00). div_val accepts only legal values (< 9), and div_en/div_val cannot change while timer_en is High; an illegal write keeps the old value and, at advanced level, returns tim_pslvrr.

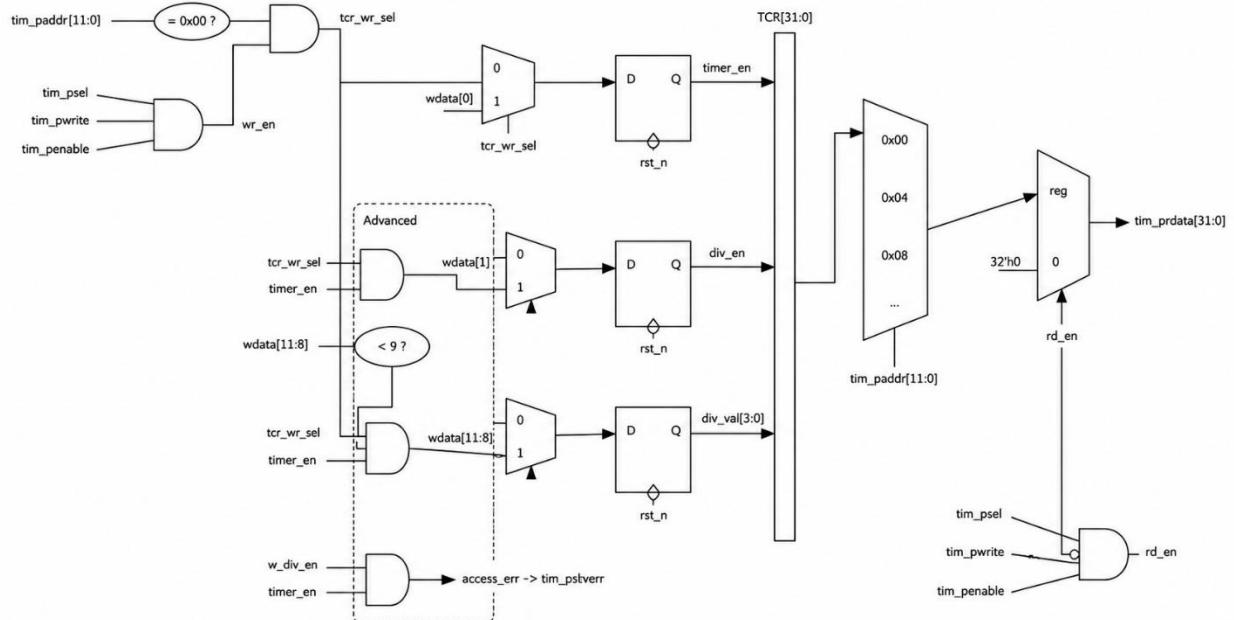


Figure 5: Logic diagram - TCR

3.2.2. Data and Compare Registers (TDR0, TDR1, TCMP0, TCMP1)

TCMP0/TCMP1 are plain 32-bit programmable registers using the generic structure: a hold/load mux selected by the register write strobe feeds a flip-flop, read back through the read mux. TDR0/TDR1 hold the live 64-bit counter value (a read returns counter_val, a write loads an initial value).

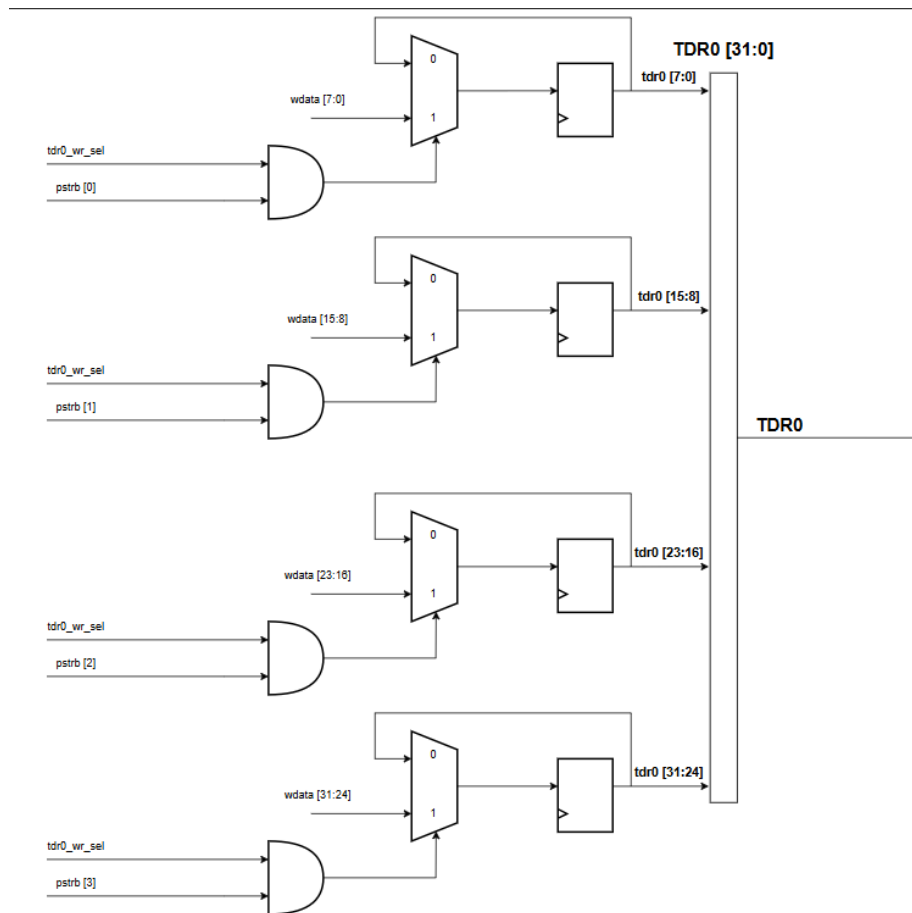


Figure 6: Logic diagram TDR0

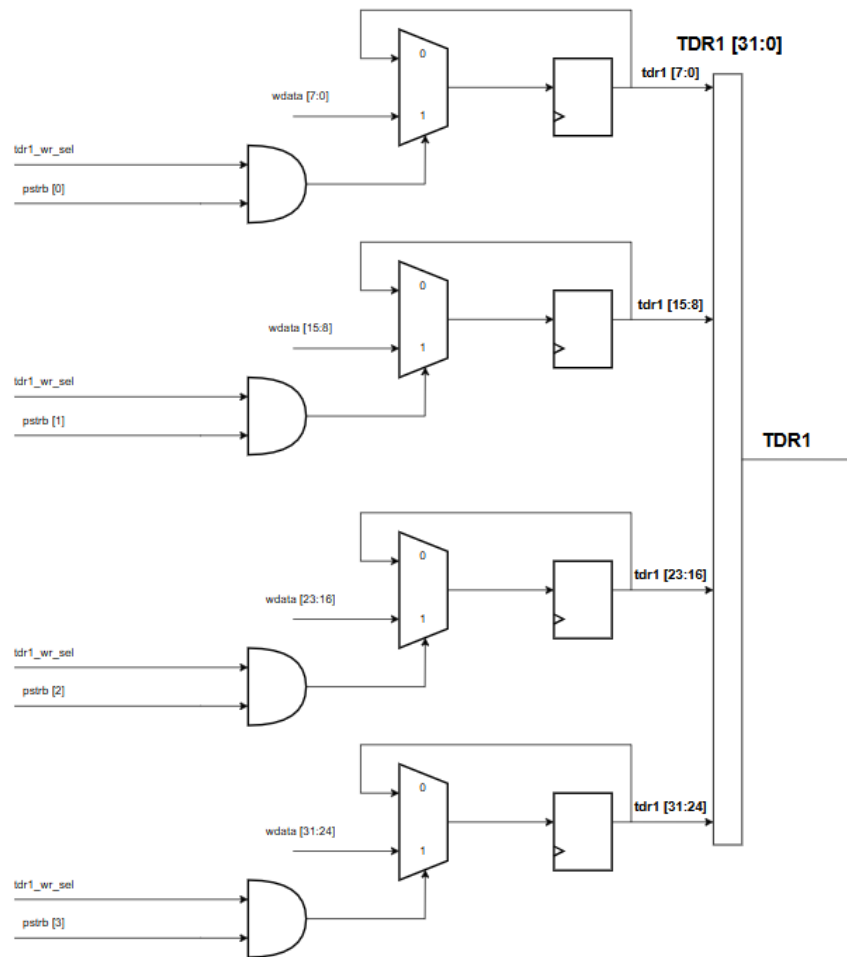


Figure 7: Logic diagram TDR1

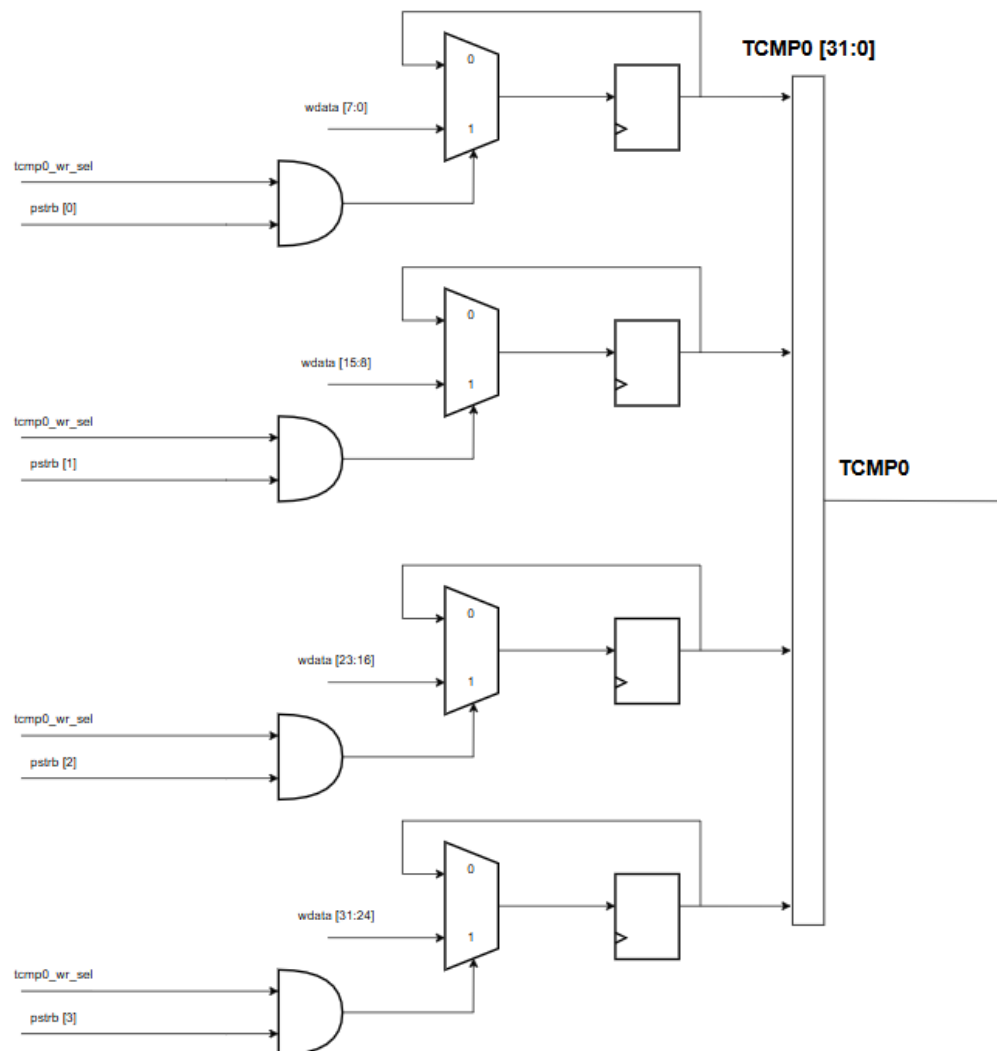


Figure 8 Logic Diagram TCMP0

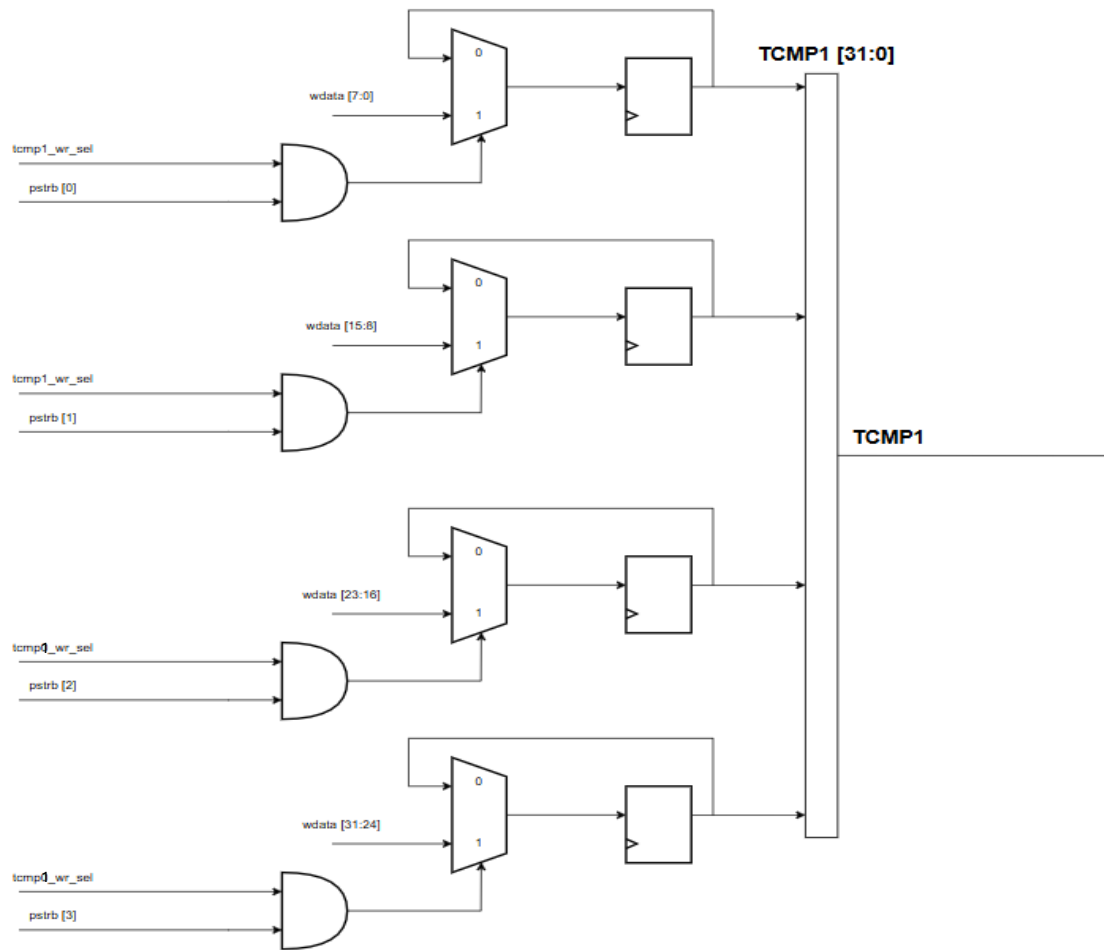


Figure 9 Logic Diagram TCMP1

3.2.3. Timer Interrupt Enable Register (TIER)

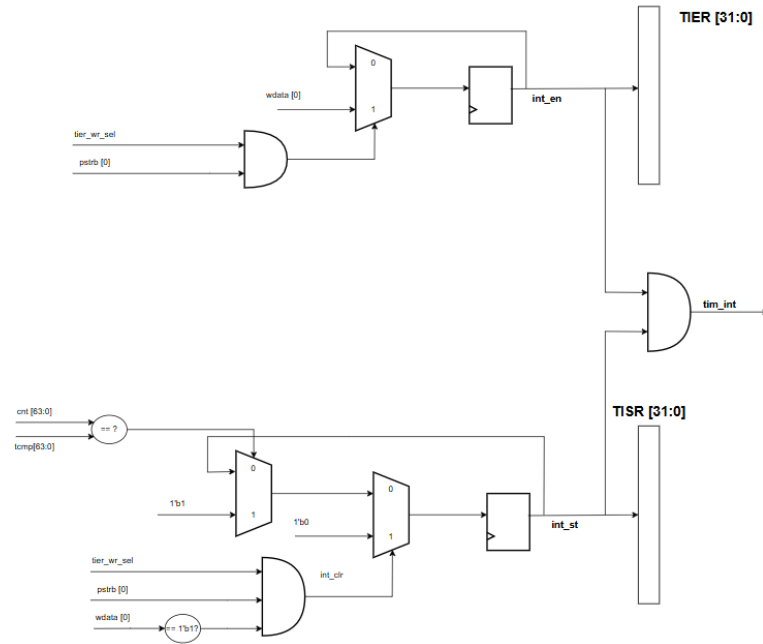


Figure 10 Logic Diagram TIER

The TIER module stores the interrupt enable bit (**int_en**). The circuit employs a single Flip-Flop to maintain this state. It selectively updates its value from **wdata** only upon receiving a valid write request (when **tier_wr_sel** AND **pstrb** are simultaneously active). This bit acts as a physical hardware gate, deciding whether the internal interrupt event is propagated to the host CPU.

3.2.4. Timer Interrupt Status Register (TISR)

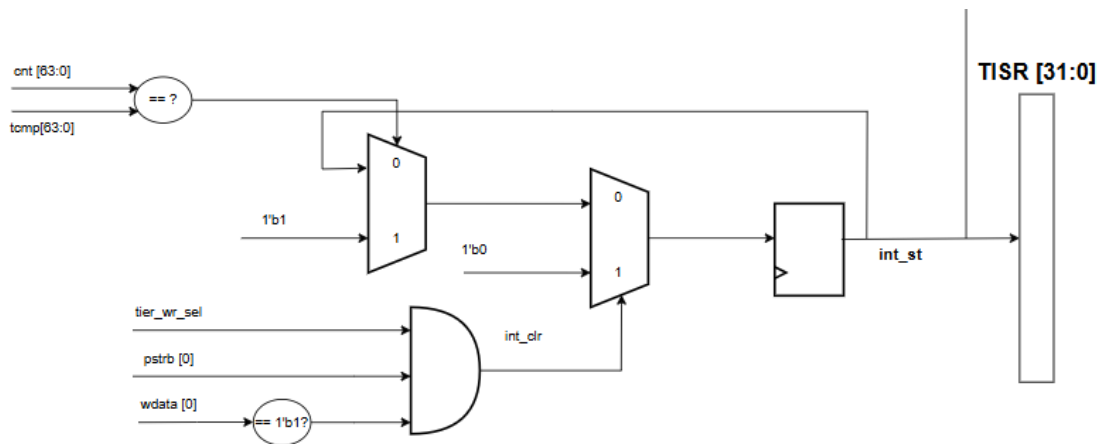


Figure 11 Logic Diagram TISR

The TISR manages the interrupt status flag (int_st), operating via a rigorous hardware-software handshake mechanism:

- Hardware Set: The flag automatically asserts to '1' the exact moment the internal comparator detects a match (cnt == tcmp).
- Software Clear (Write-1-to-Clear): To acknowledge and clear this flag, the software must write a '1' to this bit. This operation triggers the internal int_clr signal, which directs the multiplexer to forcefully reset the Flip-Flop state to '0'. Writing a '0' has no effect, preventing accidental clearances.

3.2.5. Timer Halt Control / Status Register (THCSR)

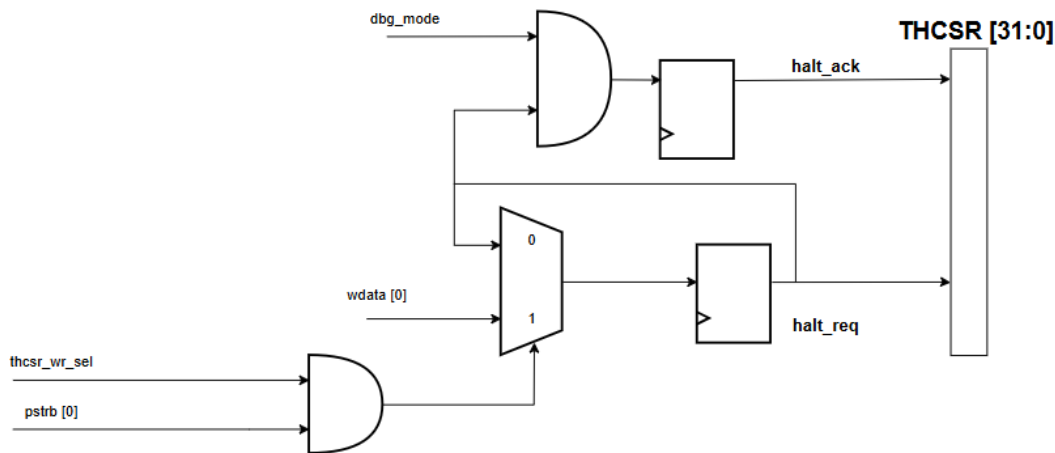


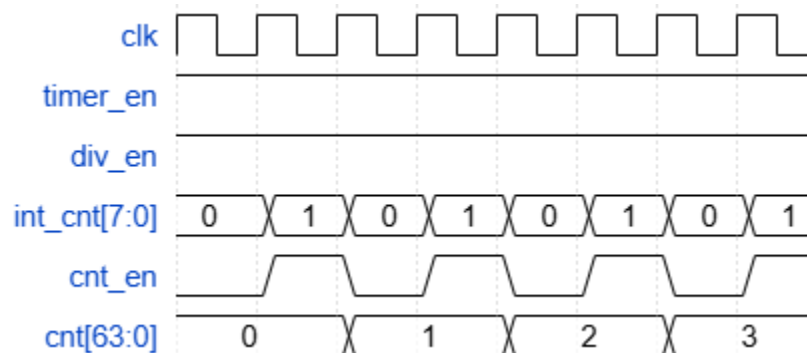
Figure 12 Logic Diagram THCSR

The THCSR logic handles the debug/halt capabilities via two dedicated signals:

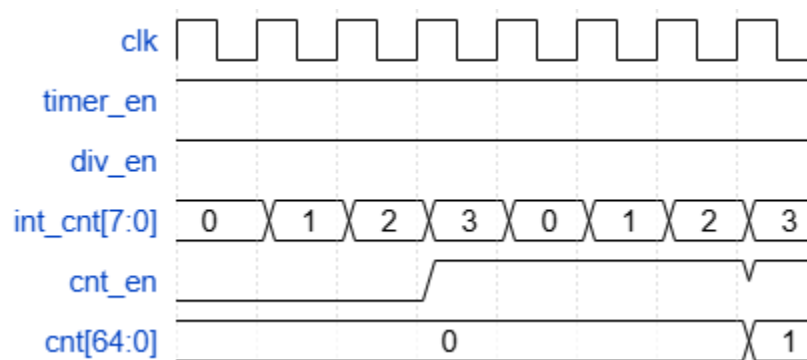
- `halt_req` (Halt Request): A software-driven control bit. A valid write operation stores the stop request in a Flip-Flop.
- `halt_ack` (Halt Acknowledge): A read-only status feedback bit. This signal asserts to '1' only when a halt request is pending (`halt_req = 1`) AND the system is externally configured to be in debug mode (`dbg_mode = 1`).

3.3. Counter control

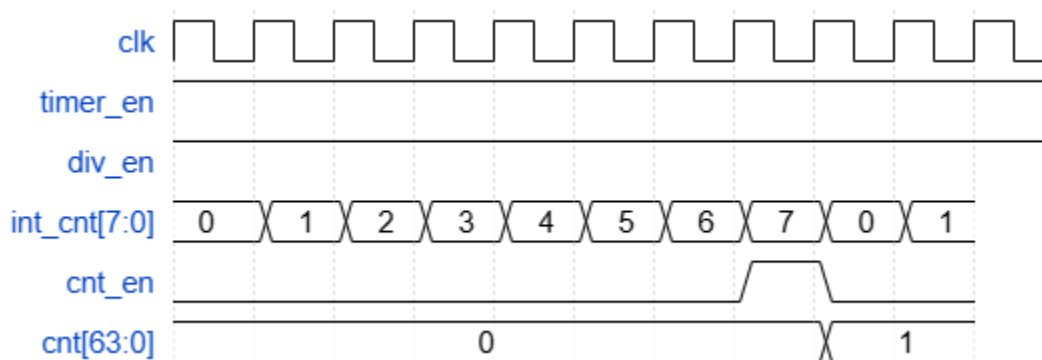
- $\text{div_val} = 1 \rightarrow \text{divide-by-2}$ (limit = 1):



- $\text{div_val} = 2 \rightarrow \text{divide-by-4}$ (limit = 3):



- $\text{div_val} = 3 \rightarrow \text{divide-by-8}$ (limit = 7):



The counter-control block generates the one-cycle counter-enable pulse `cnt_en` that paces the 64-bit counter. An 8-bit internal divider `int_cnt` counts 0..limit with $\text{limit} = 2^{\text{div_val}} - 1$; `cnt_en` is produced when the timer is enabled and either `div_en = 0` (count every clock) or `int_cnt` reaches limit. At advanced level a halt freezes `int_cnt` and masks `cnt_en`, so the counting period is preserved when counting resumes.

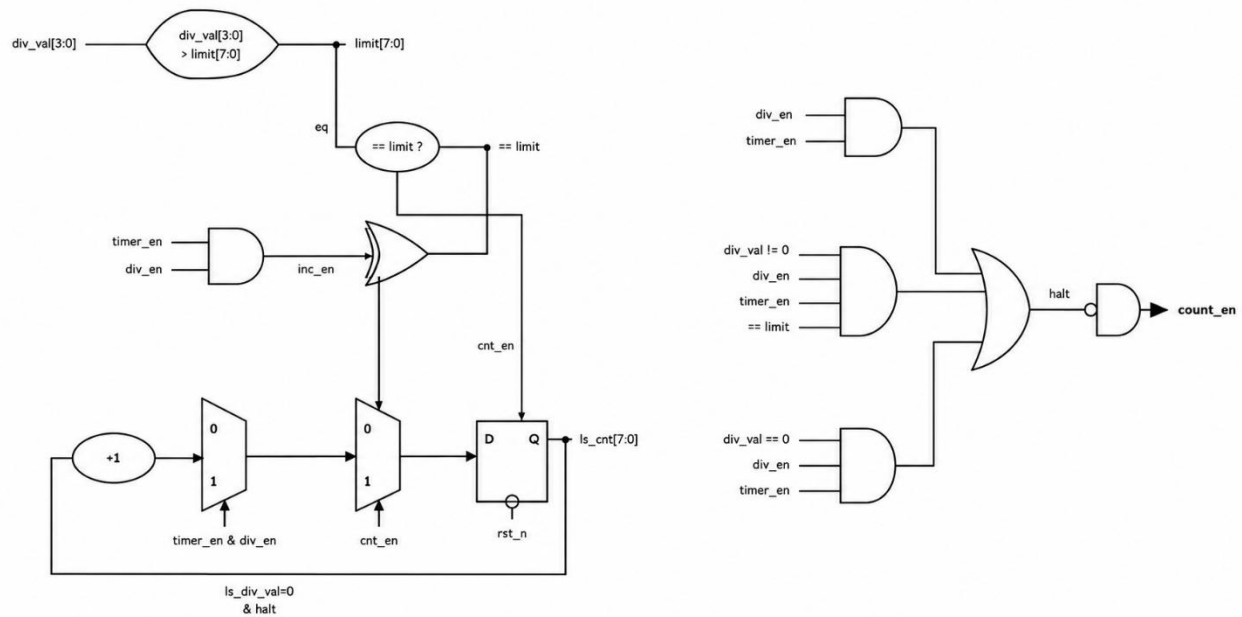


Figure 13: Logic diagram - counter control

3.4. 64-bit counter

The counter is a 64-bit up-counter formed by TDR1:TDR0. On each clock it either loads a software value (tdrX_wr_sel), increments by one (cnt_en), or holds, and it keeps counting through interrupts and through overflow. At advanced level a timer_en High-to-Low transition clears the counter in hardware (no software re-init needed); at standard level software re-initialises TDR0/TDR1 before the next run.

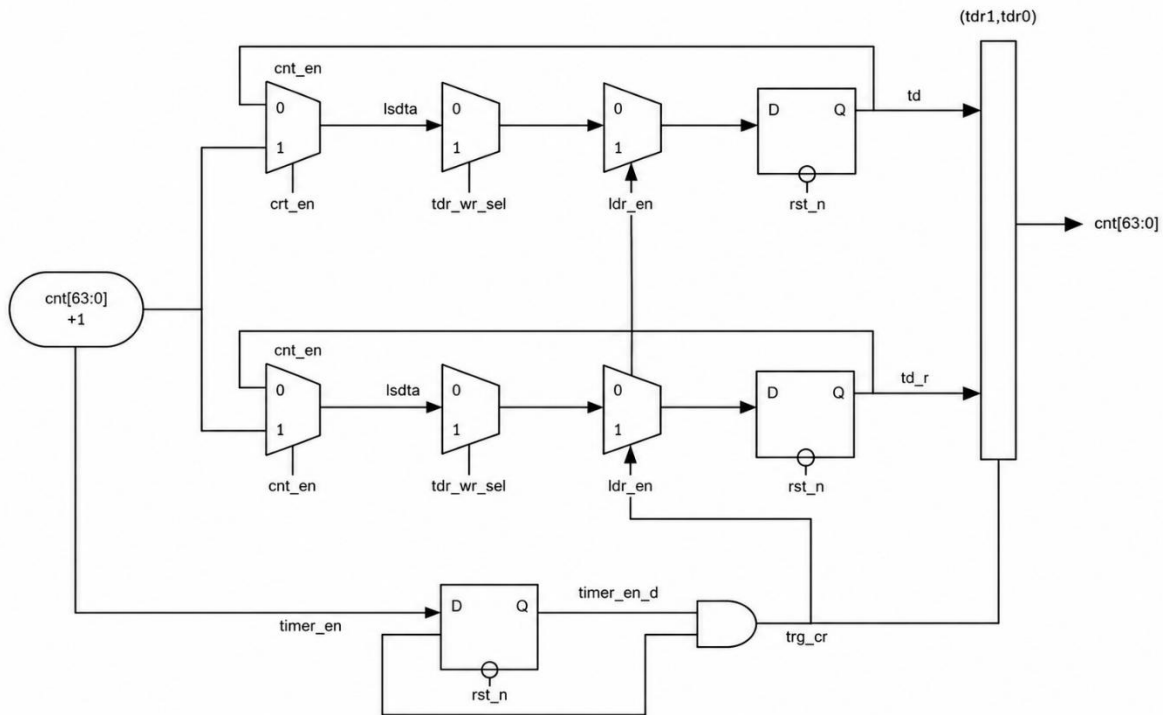
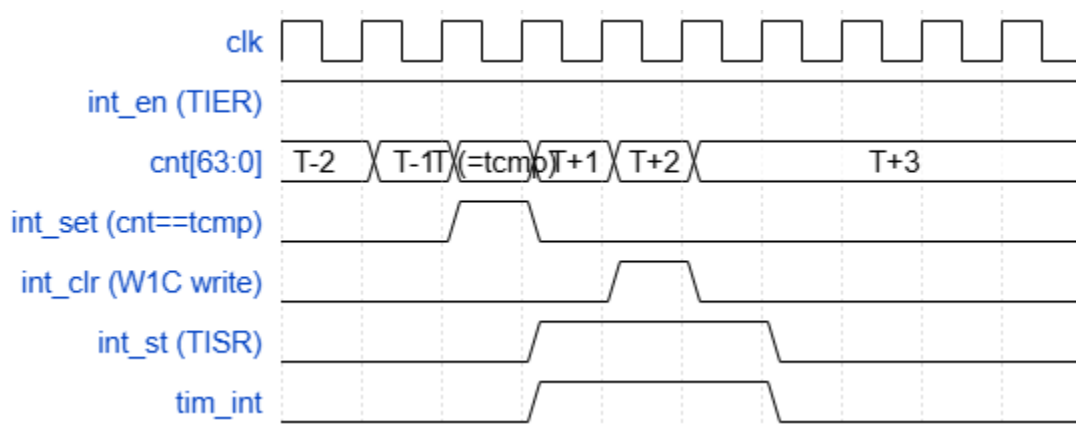


Figure 14: Logic diagram - 64-bit counter (TDR0/TDR1)

3.5. Interrupt

- waveform of interrupt:



A maskable, level interrupt. A 64-bit comparator checks $\text{counter_val} == \text{cmp_val}$ ($\{\text{TCMP1}, \text{TCMP0}\}$); a match sets the pending bit int_st , which is held until a write-1-to-clear (highest priority). The output is $\text{tim_int} = \text{int_st} \text{ AND } \text{int_en}$, so clearing TIER.int_en masks the output without clearing the pending bit. The counter keeps counting regardless of the interrupt state.

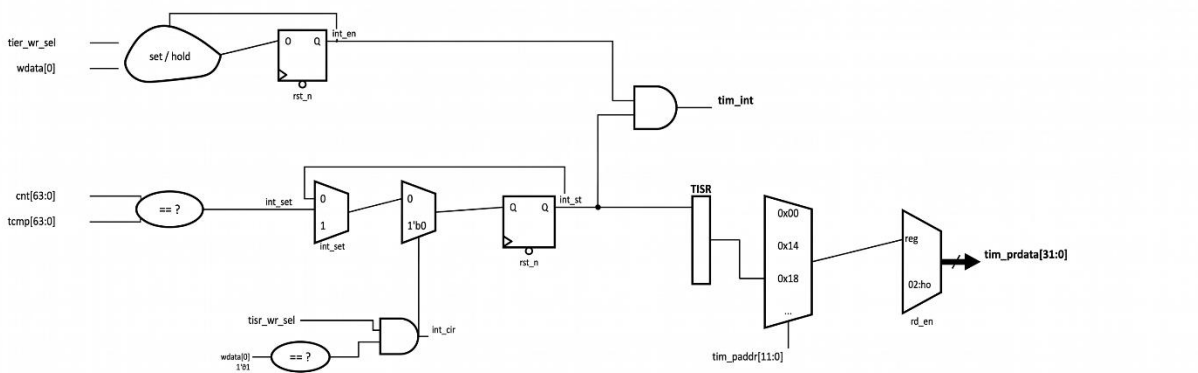
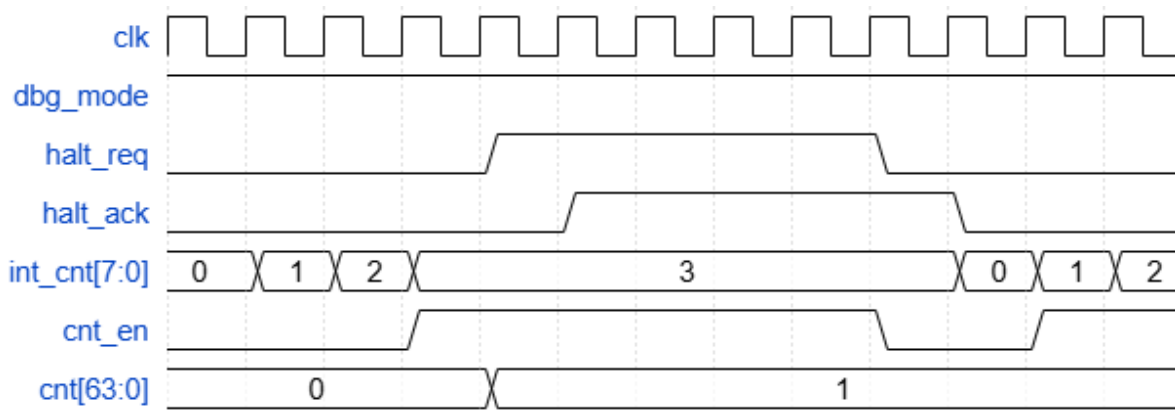


Figure 15: Logic diagram - interrupt

3.6. Halt mode (Advanced level)

- waveform of halt:



During debug the counter can be halted when `dbg_mode = 1` and `THCSR.halt_req = 1`; the timer then asserts `halt_ack`. `halt = dbg_mode AND halt_req` masks `cnt_en` and freezes the internal divider `int_cnt`, so the current count period is preserved and counting resumes seamlessly when `halt_req` returns to 0.

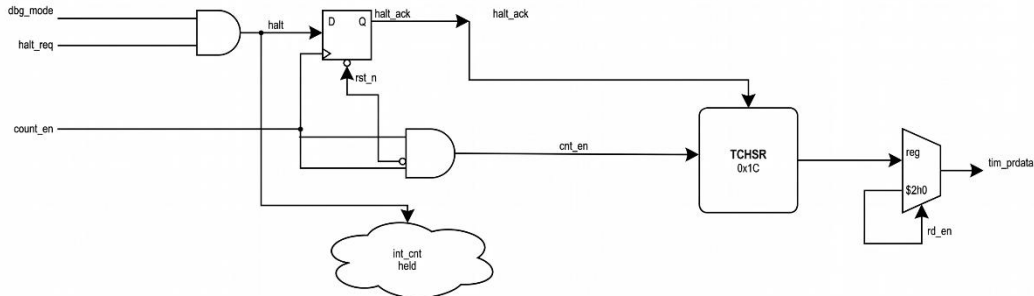


Figure 16: Logic diagram - halt control

4. History

Date	Version	Description	Author
06/06/2026	1.0	Newly created	Hoàng Ngọc Gia Bảo
10/06/2026	2.0	Make a few more improvements	Hoàng Ngọc Gia Bảo
16/06/2026	3.0	Make a few more improvements	Hoàng Ngọc Gia Bảo
26/06/2026	4.0	Final version	Hoàng Ngọc Gia Bảo